

System Presentation Reviewer and Script

GrocerEase Inventory Management System

System Presentation Reviewer and Script

System name: GrocerEase Inventory Management System (GrocerEaseIMS)

Group members and assigned modules:

Member 1: Login and Dashboard

Member 2: Category and Supplier Modules

Member 3: Products, Stocks, and Inventory Logs

Member 4: Archives, Backup and Recovery

Purpose: Guide for understanding, reviewing, and presenting only the actual implemented modules of the system.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

1. System Architecture Overview

GrocerEaseIMS is a PHP and MySQL inventory management system. The system uses PHP views for the interface, controllers for processing form/AJAX requests, model classes for database logic, and MySQL as the database.

General flow: Admin action -> PHP view or button -> form submit or fetch request -> controller -> model method -> MySQL query -> redirect or JSON response -> UI feedback.

Layer	Files	Purpose
UI Layer	views/inventory/*.php, assets/css/home.css, assets/js/pages.js	Shows pages, tables, forms, modals, filters, buttons, and SweetAlert prompts.
Controller Layer	controllers/**/*.php, controllers/backup.php	Validates requests, calls models, redirects or returns JSON.
Model Layer	models/product.php, category.php, supplier.php, Stocks.php, BackupManager.php	Contains reusable database operations.
Database Layer	config/db.php, MySQL grocer_easedb	Stores users, products, categories, suppliers, stocks, movements, and logs.

2. Member 1 Reviewer - Login and Dashboard

Login Module

The Login Module authenticates the admin before allowing access to the inventory system. The controller checks if username and password are filled in, calls the User model to authenticate, then creates session values when login is valid.

```
File: controllers/authentication.php
if (empty($username)) $error["username"] = "Username is required";
if (empty($password)) $error["password"] = "Password is required";
if (!$id=$user->AuthenticateUser($username, $password)) {
    $error["invalid"] = "Invalid username or password";
}
$_SESSION['logged_in'] = true;
$_SESSION['id'] = $id;
```

How it works: if credentials are invalid, errors are saved in session and the user returns to index.php. If valid, the system stores logged_in and id in the session, then redirects to dashboard.php.

Dashboard Module

The Dashboard summarizes the system status. It shows active products, categories, suppliers, total stocks, low stock items, out-of-stock items, backup status widgets, quick actions, and charts.

File: views/inventory/dashboard.php

System Presentation Reviewer and Script

GrocerEase Inventory Management System

```
$total_product = $product->GetTotalProducts();  
$total_categories = $category->GetTotalCategories();  
$total_suppliers = $supplier->GetTotalSupplier();  
$total_quantity = $stock->GetTotalStockQuantity();  
$low_stock = $stock->GetTotalLowStockItems();  
$out_of_stock = $stock->GetTotalOutOfStockItems();
```

The dashboard creates model objects, asks each model for totals, then displays those totals as cards. Chart data comes from stock and stock movement queries and is passed to Chart.js.

Member 1 Key Functionalities

- Login validation and session creation.
- Dashboard KPI cards for inventory status.
- Charts for stock overview, stock status, and inventory trend.
- Quick actions for common admin tasks.

Member 1 Script

Good day. I will discuss the Login and Dashboard modules. The Login module protects the system by requiring valid admin credentials before accessing inventory pages. When the user submits the login form, the controller validates the username and password, calls the User model, and creates a session if the account is valid.

After login, the user is redirected to the Dashboard. The Dashboard gives a quick overview of the system, including active products, total categories, total suppliers, stock quantity, low stock, out of stock, and backup status. These values come from the model classes and database queries.

The dashboard is important because it helps the admin quickly understand the current inventory status and decide what action to take next.

Q: Why is login important?

A: It prevents unauthorized users from accessing inventory records.

Q: What does the dashboard show?

A: It shows inventory totals, stock warnings, backup status, quick actions, and charts.

Q: How does the dashboard get data?

A: It calls model methods that query the MySQL database.

3. Member 2 Reviewer - Category and Supplier Modules

Category Module

The Category Module organizes products into groups. It supports adding, viewing, editing, soft deleting, restoring, and hard deleting categories. Category data is used by products and dashboard reports.

```
File: models/category.php  
public function AddCategory($name, $description)  
public function CheckDuplicateCategory($category_name)  
public function GetPaginatedCategories($page = 1, $limit = 10)
```

System Presentation Reviewer and Script

GrocerEase Inventory Management System

```
public function SoftDeleteCategory($id)
public function RestoreCategory($id)
```

Important logic: before adding a category, the system checks duplicate category names where `is_deleted = 0`. Deleting a category is a soft delete, and products connected to that category are set to NULL to avoid broken category references.

File: models/category.php

```
UPDATE products SET category_id_fk = NULL WHERE category_id_fk = :id
UPDATE categories SET is_deleted = 1, deleted_at = NOW() WHERE category_id_pk = :id
```

Supplier Module

The Supplier Module manages supplier records such as supplier name, contact person, phone number, email, address, and company name. Suppliers can be added, viewed, edited, soft deleted, restored, and hard deleted.

File: models/supplier.php

```
public function AddSupplier($name, $contact_person, $phone_number, $email, $address,
$company_name)
public function CheckDuplicateSupplier($supplier_name)
public function EditSupplier($id, $name, $contact_person, $phone_number, $email, $address,
$company_name)
public function SoftDeleteSupplier($id)
```

Supplier soft delete also checks product_supplier relationships. If a supplier is connected to products, the relationship is updated to NULL before marking the supplier as deleted.

Member 2 Key Functionalities

- Category add, edit, view, search/pagination, delete, restore.
- Supplier add, edit, view, search/pagination, delete, restore.
- Duplicate checking for categories and suppliers.
- Soft delete protects records from immediate permanent removal.

Member 2 Script

I will explain the Category and Supplier modules. The Category module is used to organize products into groups. The admin can add, view, edit, and delete categories. The system also checks if a category already exists to avoid duplicate records.

When a category is deleted, the system does not immediately remove it permanently. It marks the category as deleted and updates related products so their category reference will not break.

The Supplier module stores supplier information such as supplier name, contact person, phone number, email, address, and company name. It also supports add, edit, delete, restore, and duplicate checking.

These modules are important because products depend on categories and suppliers for proper organization and purchasing information.

Q: Why do we need categories?

A: Categories organize products and make inventory easier to manage.

Q: What happens when a category is deleted?

System Presentation Reviewer and Script

GrocerEase Inventory Management System

A: It is soft deleted and connected products are updated to avoid broken references.

Q: What information is stored for suppliers?

A: Supplier name, contact person, phone, email, address, and company name.

4. Member 3 Reviewer - Products, Stocks, and Inventory

Product Module

The Product Module manages product records. It includes add product, view list, edit product, soft delete, search, pagination, product status, category link, preferred supplier display, selling price, and description.

```
File: controllers/products/add_product_process.php
if (empty($product_name) || empty($category) || empty($selling_price) || $status === '')
    $errors['form'] = "Please fill in all required fields.";
if ($product->CheckDuplicateProduct($product_name, $category))
    $errors['product_name'] = "Product already exists in the selected category.";
$product_id = $product->AddProduct($product_name, $category, $selling_price, $description,
$status);
$stock->AddProductStock($product_id, 0, date('Y-m-d H:i:s'));
```

Add Product validates the form, checks duplicate product within the selected category, inserts the product, then creates a stock row with quantity 0.

```
File: models/product.php
UPDATE products SET is_deleted = 1, deleted_at = NOW() WHERE product_id_pk = :id
```

Delete Product is a soft delete. The product remains in the database but is hidden from the active list.

Stocks Module

The Stocks Module manages product quantities. Stock In adds quantity, Stock Out subtracts quantity, and the system prevents invalid or excessive stock-out quantities.

```
File: models/Stock/Stocks.php
public function StockIn($product_id, $quantity, $date)
UPDATE stocks set quantity = quantity + :quantity, last_updated = :date WHERE product_id_fk = :id

public function StockOut($product_id, $quantity, $date)
UPDATE stocks set quantity = quantity - :quantity, last_updated = :date WHERE product_id_fk = :id
AND quantity >= :quantity
```

Inventory Logs

Inventory logs are created from stock movements. The stock_movements table records quantity, reference type IN/OUT, reference ID, reason, date, and product ID. This gives traceability for stock changes.

```
File: models/Stock/stockmovements.php
INSERT INTO stock_movements (quantity, reference_type, reference_id, reason, date, product_id)
VALUES(:quantity, :type, :ref_id, :reason, :date, :product_id)
```

System Presentation Reviewer and Script

GrocerEase Inventory Management System

Member 3 Key Functionalities

- Product add, edit, delete, search, pagination, and status display.
- Stock In and Stock Out with quantity validation.
- Inventory logs and trend reporting from stock_movements.
- Transaction logs record stock changes.

Member 3 Script

I will discuss Products, Stocks, and Inventory Logs. The Product Module allows the admin to add, view, edit, delete, and search products. When adding a product, the controller validates required fields and duplicate product names. After saving the product, the system also creates a stock record with zero quantity.

For stock management, Stock In increases the product quantity, while Stock Out decreases it. The system validates the quantity and prevents Stock Out if the requested quantity is greater than the current stock.

Every stock movement is recorded in inventory logs with reference type, reference ID, quantity, reason, date, and product. This makes the inventory changes traceable.

Q: What happens after adding a product?

A: The system inserts the product and creates an initial stock record with quantity 0.

Q: How does Stock Out prevent negative stock?

A: It checks current quantity and the SQL update requires quantity >= requested quantity.

Q: Why are inventory logs important?

A: They show the history and reason behind stock changes.

5. Member 4 Reviewer - Archives, Backup and Recovery

Archives Module

The Archives Module displays deleted products, categories, and suppliers. It lets the admin restore archived records or permanently delete them when needed. The page uses tabs for products, categories, and suppliers.

```
File: views/inventory/archived.php
$tab = $_GET['tab'] ?? 'products';
if ($tab == 'products') {
    $products = $product->GetDeletedProductsPaginated($page, $limit);
} elseif ($tab == 'categories') {
    $category = $categories->GetDeletedCategoriesPaginated($page, $limit);
} else {
    $supplier = $suppliers->GetDeletedSuppliersPaginated($page, $limit);
}
```

The archive page checks the selected tab, loads deleted records using the correct model, and includes the matching deleted record table.

- Products archive uses Product::GetDeletedProductsPaginated and Product::RestoreProduct.
- Categories archive uses Category::GetDeletedCategoriesPaginated and

System Presentation Reviewer and Script

GrocerEase Inventory Management System

Category::RestoreCategory.

- Suppliers archive uses Supplier::GetDeletedSuppliersPaginated and Supplier::RestoreSupplier.
- Hard delete is available in separate controllers, but should be used carefully because it permanently removes data.

Backup and Recovery Module

Backup and Recovery protects system data. The implemented user-facing features are Create Backup, Restore Backup, View Backup History, Download Backup, and Delete Backup. Backups are stored as SQL files in the local /backups folder.

- Main UI: views/inventory/settings.php.
- Controller: controllers/backup.php.
- Model: models/BackupManager.php.
- Access: controller requires logged-in session.

Create Backup Flow

1. Admin opens Backup and Recovery.
2. Admin clicks Backup now.
3. Controller checks login and POST method.
4. BackupManager reads database tables.
5. System writes table structure and data into a SQL file.
6. System validates the SQL file.
7. UI shows success or error and refreshes history.

```
File: models/BackupManager.php
$tables = $this->getTables();
foreach ($tables as $table) {
    $this->dumpTable($handle, $table);
}
$validation = $this->validateBackupFile($filepath);
```

Restore Flow

1. Admin uploads/selects a valid SQL backup file.
2. System validates extension, readability, size, and SQL-like content.
3. Admin confirms restore warning.
4. System creates pre_restore safety backup before changing data.
5. System executes SQL statements into the database.
6. UI shows restore complete or restore failed.

```
File: models/BackupManager.php
$validation = $this->validateBackupFile($filepath, $sourceName);
$safetyBackup = $this->createBackup('pre_restore');
$statements = $this->splitSqlStatements($sql);
$this->conn->exec('SET FOREIGN_KEY_CHECKS=0');
foreach ($statements as $statement) { $this->conn->exec(trim($statement)); }
$this->conn->exec('SET FOREIGN_KEY_CHECKS=1');
```

Important explanation: a full backup is a manual saved database state. A safety backup is created right before restore and contains the current state before applying the selected

System Presentation Reviewer and Script

GrocerEase Inventory Management System

backup.

History, Download, and Delete

File: controllers/backup.php

```
case 'list': respondJson(['status' => 'success', 'data' => $manager->getBackupList()]);
case 'download': $filepath = $manager->resolveBackupPath($filename); readfile($filepath);
case 'delete': respondJson($manager->deleteBackup($filename));
```

History scans /backups/*.sql and returns filename, date, type, size, and status. Download streams the file. Delete validates the filename path first, then removes the selected backup file.

Security and Common Errors

- Only logged-in users can call backup controller actions.
- Only .sql backup files are accepted for restore.
- File paths are validated using basename, regex, realpath, and folder containment.
- Delete requires confirmation and POST request.
- Common error: Unexpected token < means the frontend expected JSON but received HTML, often login page, 404, or PHP error.
- Common error: 404 means wrong controller path or missing controller file.

Member 4 Script

I will discuss the Archives Module and Backup and Recovery Module. The Archives Module stores records that were soft deleted from Products, Categories, and Suppliers. Instead of immediately removing records, the system marks them as deleted, then displays them in the archive page.

In the archive page, the admin can switch tabs between products, categories, and suppliers. The system loads the deleted records based on the selected tab. The admin can restore a record if it was deleted by mistake, or permanently delete it if it is no longer needed.

Next is Backup and Recovery. This module protects the database. The admin can create a full SQL backup, restore a valid backup, view backup history, download backup files, and delete old backup files.

When the admin clicks Backup now, the button sends a request to controllers/backup.php. The controller calls BackupManager, which reads all database tables and writes the table structure and data into an SQL file in the backups folder.

For restore, the system first validates the selected SQL file. The admin must confirm the restore warning. Before restoring, the system creates a pre_restore safety backup. Then it executes the SQL statements into the database and shows whether the restore succeeded or failed.

This module is important because it protects the system from accidental data loss and gives the admin a way to recover records after wrong changes.

Q: What is the purpose of archives?

A: To keep soft-deleted records available for restore or permanent deletion.

Q: What is the difference between full backup and safety backup?

System Presentation Reviewer and Script

GrocerEase Inventory Management System

A: Full backup is manually created as a saved state. Safety backup is created right before restore to save the current state.

Q: Why is restore restricted?

A: Restore can overwrite database data, so only authorized users should do it.

Q: Where are backups stored?

A: In the local /backups folder as .sql files.

6. Code Reviewer Summary

Member	Modules	Key files	Main database action
1	Login, Dashboard	controllers/authentication.php, dashboard.php	Read users, read dashboard counts/charts.
2	Category, Supplier	models/category.php, models/supplier.php	Insert, update, soft delete, restore categories and suppliers.
3	Products, Stocks, Inventory	models/product.php, Stocks.php, stockmovements.php	Product CRUD, stock quantity updates, movement logging.
4	Archives, Backup/Recovery	archived.php, BackupManager.php, backup.php	Load deleted records, export/import SQL, manage backup files.

7. Final Group Presentation Flow

1. Member 1 opens: system introduction, login, authentication, and dashboard overview.
2. Transition: "Next, Member 2 will explain how records are organized through categories and suppliers."
3. Member 2 explains Category and Supplier modules, including add, edit, delete, restore, and duplicate checks.
4. Transition: "Next, Member 3 will explain the main inventory operations: products, stocks, and logs."
5. Member 3 explains Product Module, Stock In/Out, and Inventory Logs.
6. Transition: "Finally, Member 4 will explain archives and how the system protects data through backup and recovery."
7. Member 4 explains Archives and Backup and Recovery.
8. Final closing: "Overall, GrocerEaseIMS helps administrators manage grocery inventory records, monitor stock, maintain records, and protect data through backup and recovery. Thank you."

8. Final Summary

The presentation is now divided according to the actual system modules. Member 1 focuses on secure access and dashboard monitoring. Member 2 focuses on organizing records through categories and suppliers. Member 3 focuses on product and stock operations. Member 4 focuses

System Presentation Reviewer and Script

GrocerEase Inventory Management System

on archives and database protection through backup and recovery.

Each member should study the purpose, user action, controller/model logic, database action, and expected output of their assigned modules.